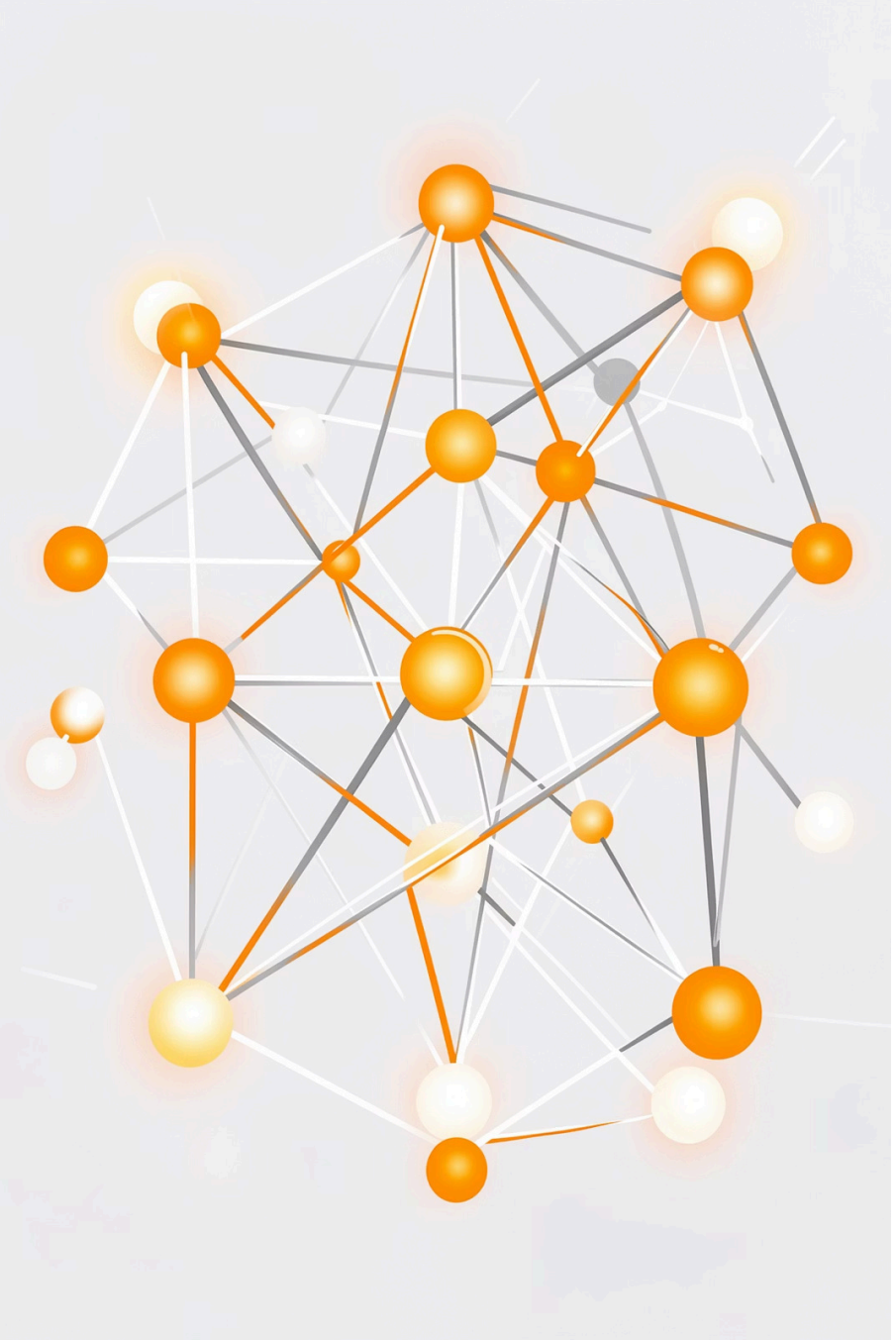


DiskANN Vamana Index

Implementation, Optimization, and Analysis on SIFT1M – 1M vectors, 128 dimensions, 10K queries

DA24B004 DA24B033 DA24B049

Benchmark & Baseline

L	Recall@10	Avg Dist Cmps	Avg Latency (μs)	P99 Latency (μs)
10	0.7820	653.4	174.4	862.4
20	0.8900	890.3	252.6	1279.8
30	0.9334	1108.3	300.2	852.8
50	0.9661	1517.4	450.7	1978.4
75	0.9818	1995.4	627.8	2672.4
100	0.9883	2444.3	760.0	2802.1
150	0.9936	3279.8	1084.3	4183.8
200	0.9960	4054.0	1427.4	5076.8

Table 2: Baseline search results on SIFT1M ($K = 10$, $R = 32$, $L_{\text{build}} = 75$, $\alpha = 1.2$, $\gamma = 1.5$).

SIFT1M Setup

1M × 128-dim float32 vectors · 10K queries · top-100 ground truth · Windows MSVC C++

Finding best configuration for Baseline:

L(build), L(search), alpha-rng parameter.

alpha = 1.2

pruning threshold

L = 100

Build list

93.3s

Build time

33.1

Avg out-degree

Search Beam Width Sweep on SIFT1M:

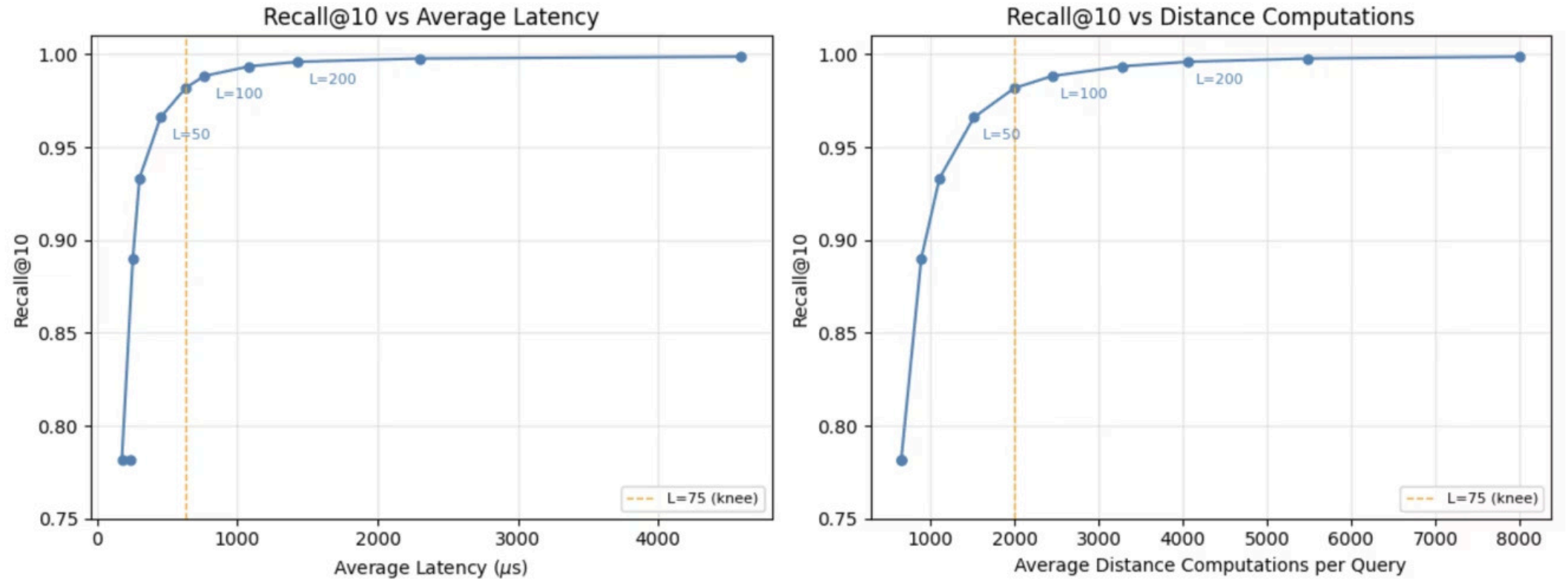


Figure 1: Recall@10 vs average latency (left) and recall@10 vs average distance computations per query (right), across $L_{\text{search}} \in \{5, 10, 20, 30, 50, 75, 100, 150, 200, 300, 500\}$. The orange dashed line marks $L = 75$.

Build Beam Width Sweep on SIFT1M:

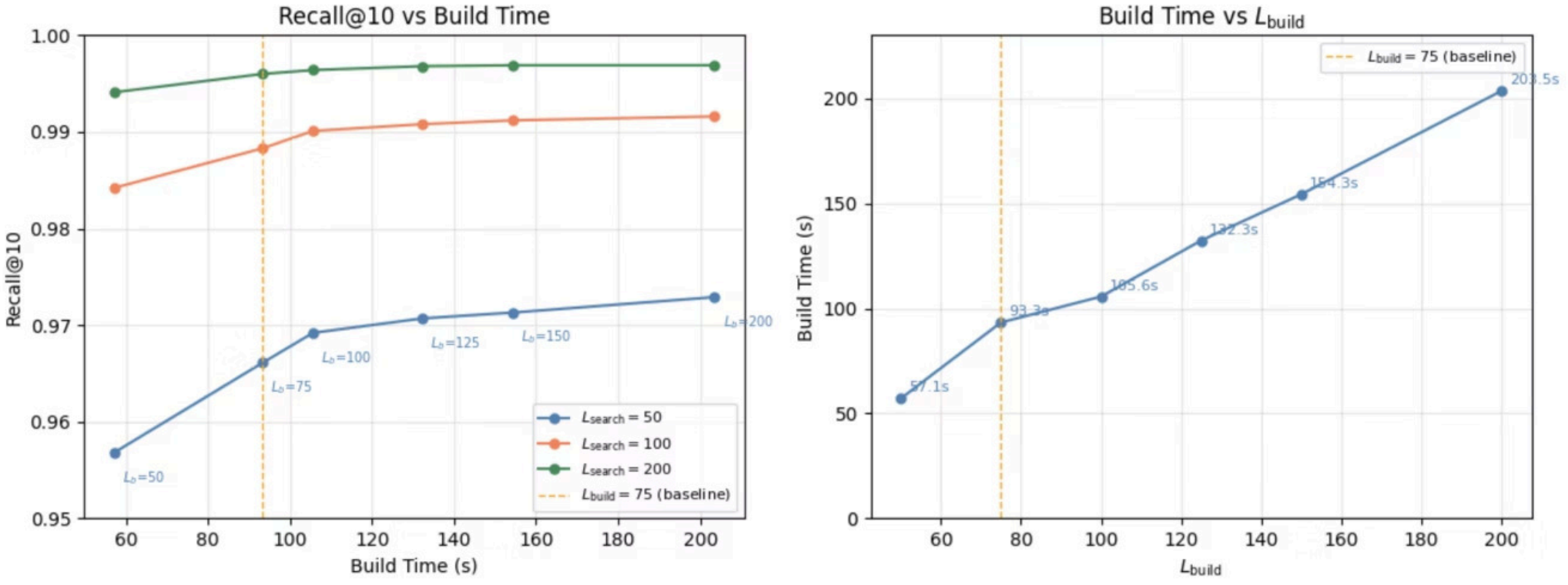


Figure 2: Left: Recall@10 vs build time for three values of L_{search}, showing diminishing recall gains beyond the baseline. Right: build time vs L_{build}, showing linear growth in build cost. The orange dashed line marks the baseline configuration ($L_{\text{build}} = 75$, 93.3s).

Alpha parameter sweep (SIFT1M, L = 75)

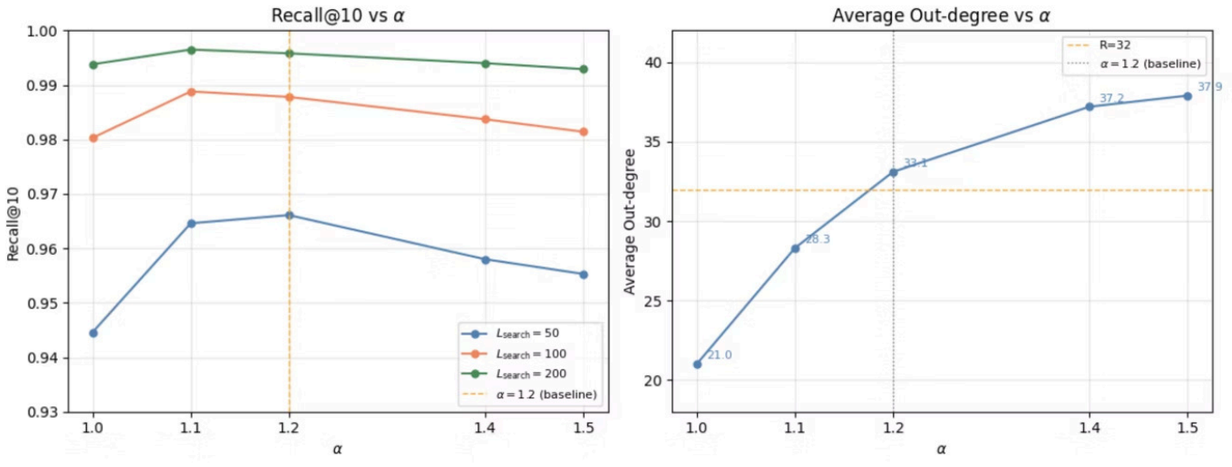


Figure 3: Left: Recall@10 vs α for three values of L_{search}. Right: average out-degree vs α. The orange dashed line on the left marks the baseline ($\alpha = 1.2$). The orange dashed line on the right marks $R = 32$.

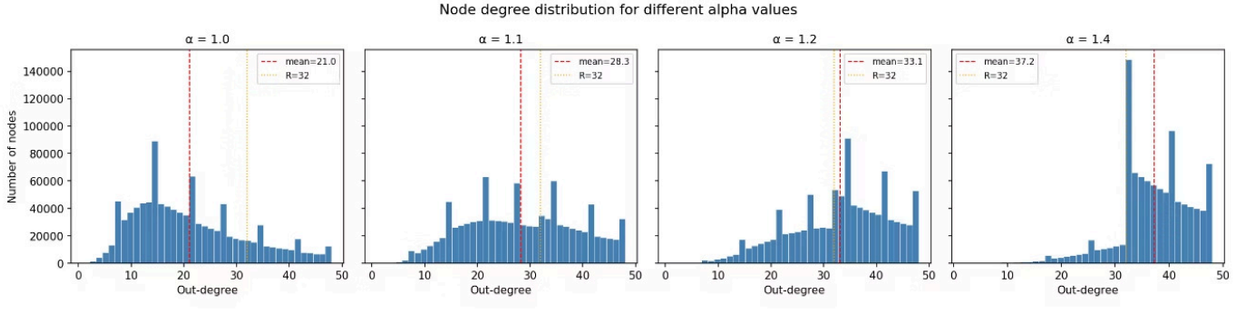


Figure 4: Node degree distribution for $\alpha \in \{1.0, 1.1, 1.2, 1.4\}$. The red dashed line marks the mean degree and the orange dotted line marks $R = 32$.

- **Recall is strictly optimized at alpha = 1.1:**
- Lower values -> overly aggressive pruning.
- Higher values -> retain redundant edges that point in similar directions.

- **The graph has a bimodal degree distribution that bottlenecks at high alpha:** sparse nodes in dense dataset regions (first bump); dense nodes accumulating backward edges (second bump).

PROPOSALS A–D

Four Search-Time Optimizations

Flat Sorted Vector

Replace `std::set` (Red-Black Tree) with a flat `std::vector` using `memmove`. Contiguous memory keeps iteration within CPU cache lines, eliminating heap allocation and pointer chasing.

Quantized ADC

Compress float32 (488 MB) → uint8 (128 MB) via scalar block quantization. Query stays float32 but data is uint8. Full re-ranking preserves recall. Latency savings: **0.8–21%**.

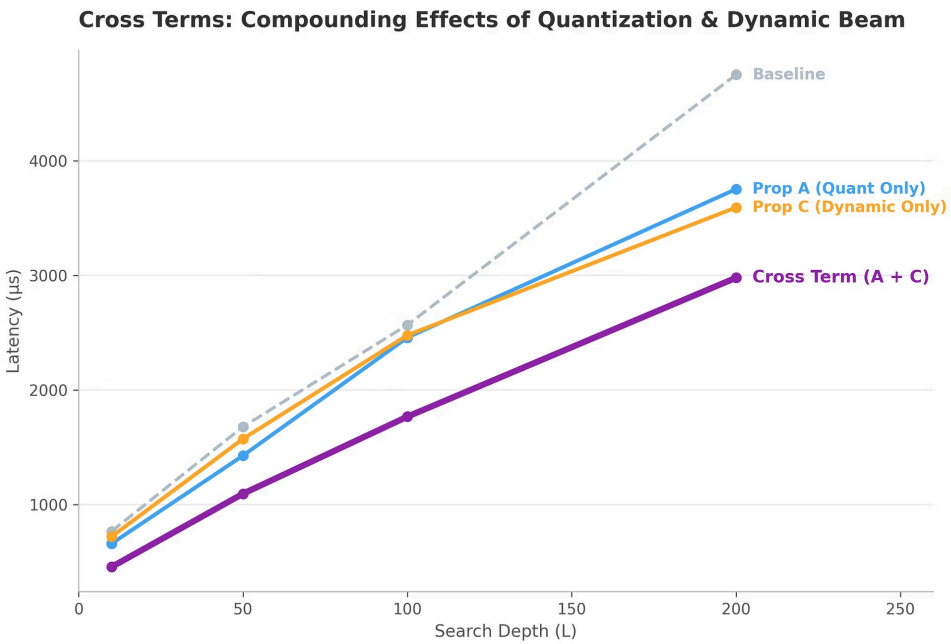
Early Abandonment

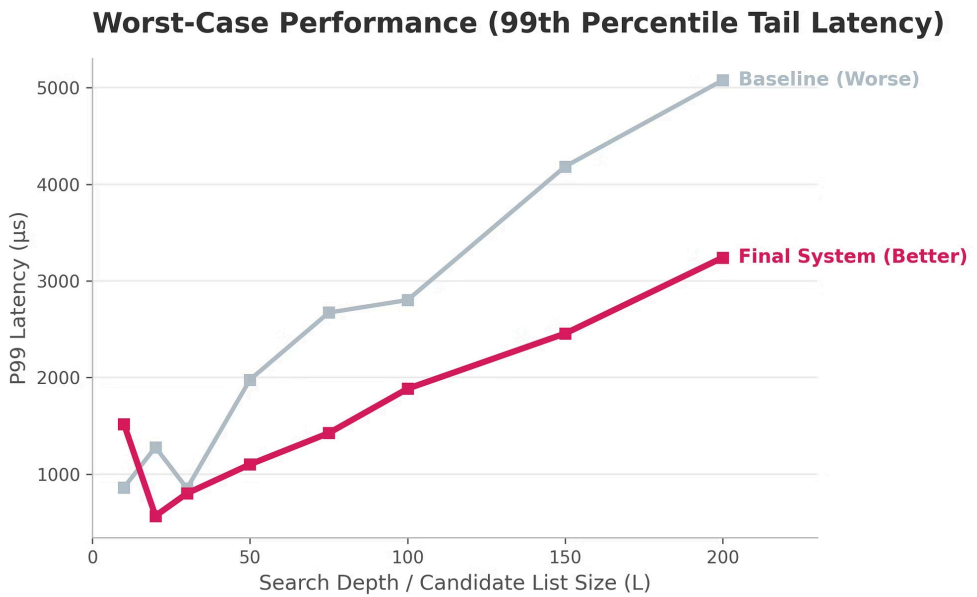
Exit distance computation early if running sum exceeds beam threshold. Checks every 16 dimensions (AVX2/AVX-512 aligned). Latency savings: **1.9–6.5%** at zero algorithmic cost.

Dynamic Beam Width

Adapt active list size *L* during traversal. Expand on stalls, contract on steady progress. Largest single gain: **up to 30.9%** latency reduction (quantized mode).

Proposals A–D: Performance Results





BUILD-TIME FIXES

Algorithmic Correctness Fixes

Medoid Initialization

Replace random entry point with the dataset point nearest to the mean vector - computed in O(n). Eliminates pathological queries from poor start regions.

Random R-Regular Pre-Init

Bootstrap every node with R random neighbors before construction, ensuring early insertions search a meaningful graph for higher-quality edges.

Full Visited Set → RobustPrune

Pass the complete visited set V (2–4× larger than top-L) to RobustPrune. This gives precisely the long-range edges that improve navigability, acting as a sample of the entire dataset

Combined effect: **60% reduction in P99 latency** at L=75 (2672 µs → 1056 µs) and **25% lower average latency**.

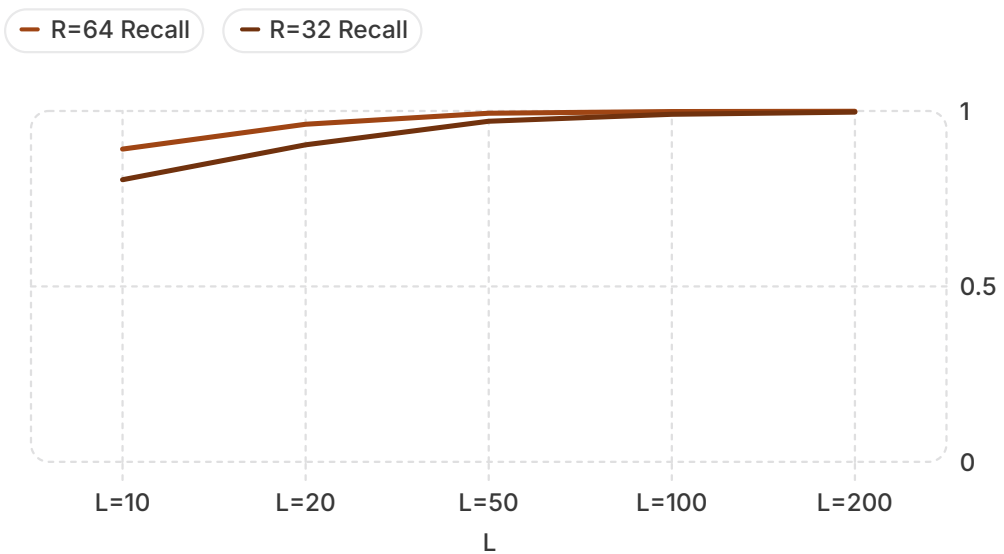
Best Configuration: Equivalent-Recall Comparison

Comparison is at **equivalent recall** - a denser graph reaches the same recall at a lower L value.

Recall	Config	L	Avg Latency (µs)	P99 Latency (µs)
≈0.982	Baseline	75	627.8	2672.4
≈0.982	**Best**	**30**	**359.6**	**801.7**
0.9936	Baseline	150	1084.3	4183.8
0.9936	**Best**	**50**	**521.7**	**1102.1**

At 0.9936 recall: **43–52% lower average latency** and **70–74% lower P99 latency** using significantly fewer distance computations.

Higher Graph Degree: R = 64



Why R=64 Wins

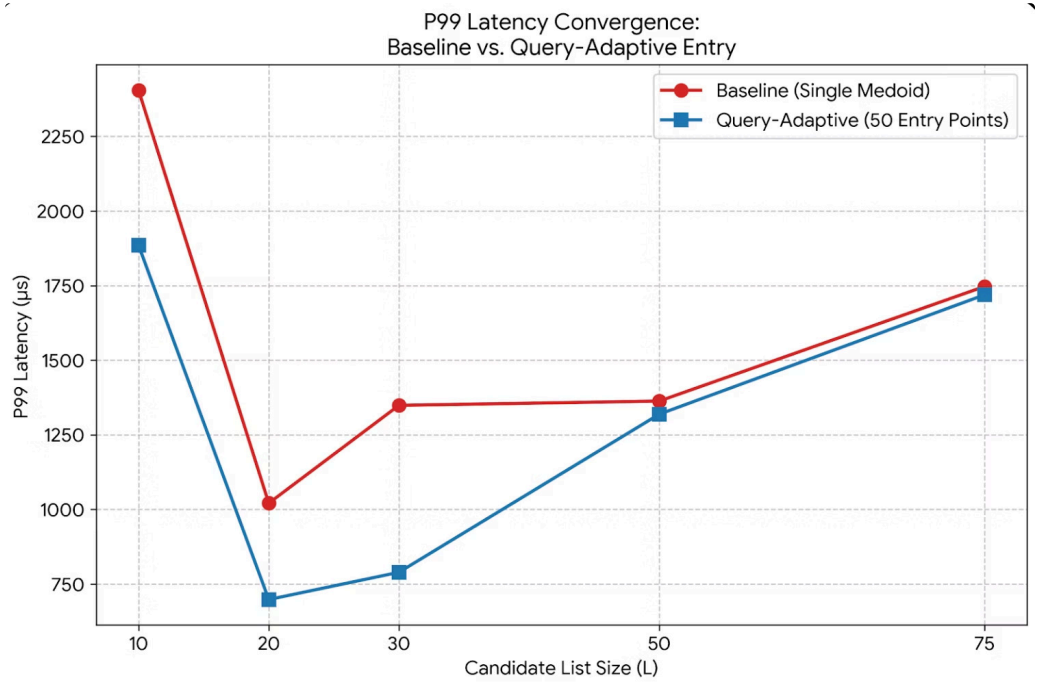
Doubling graph degree gives RobustPrune more diverse edges per node, covering more directions in embedding space. Fewer hops needed to reach true nearest neighbors.

Trade-offs

- Recall improves by **+0.002–+0.087** across all L values
- Build time: 110s (R=32) → 338s (R=64) - a one-time cost
- Per-query latency 64–73% higher at same L, but lower L suffices

Searched with **Quantized ADC** to keep per-query latency competitive.

Multi-entry points (*DiskANN++ 2023 by Ni et al.*)



Initialized using K-medoids, we choose closest point to given query and start from here.

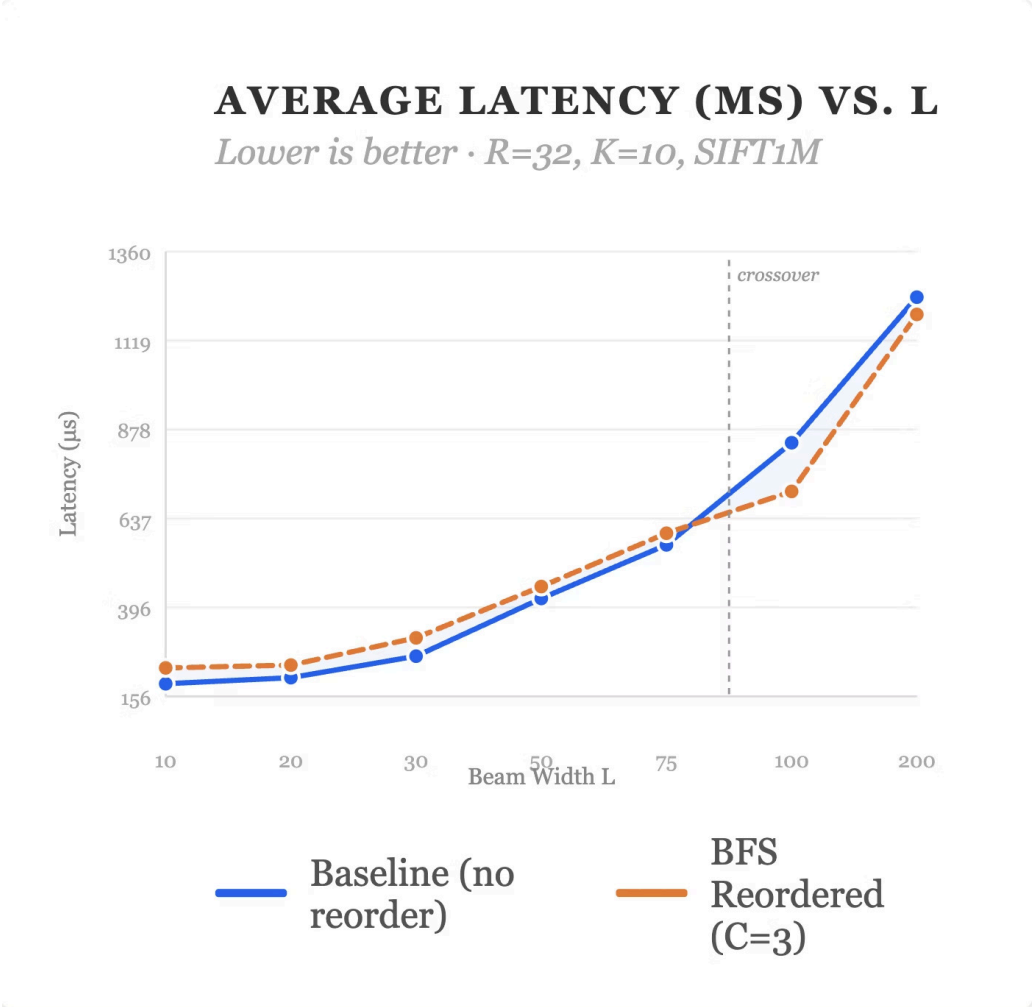
Motivation: Multi point entry did not give results we expected, so we tried scraping for other initialization ideas.

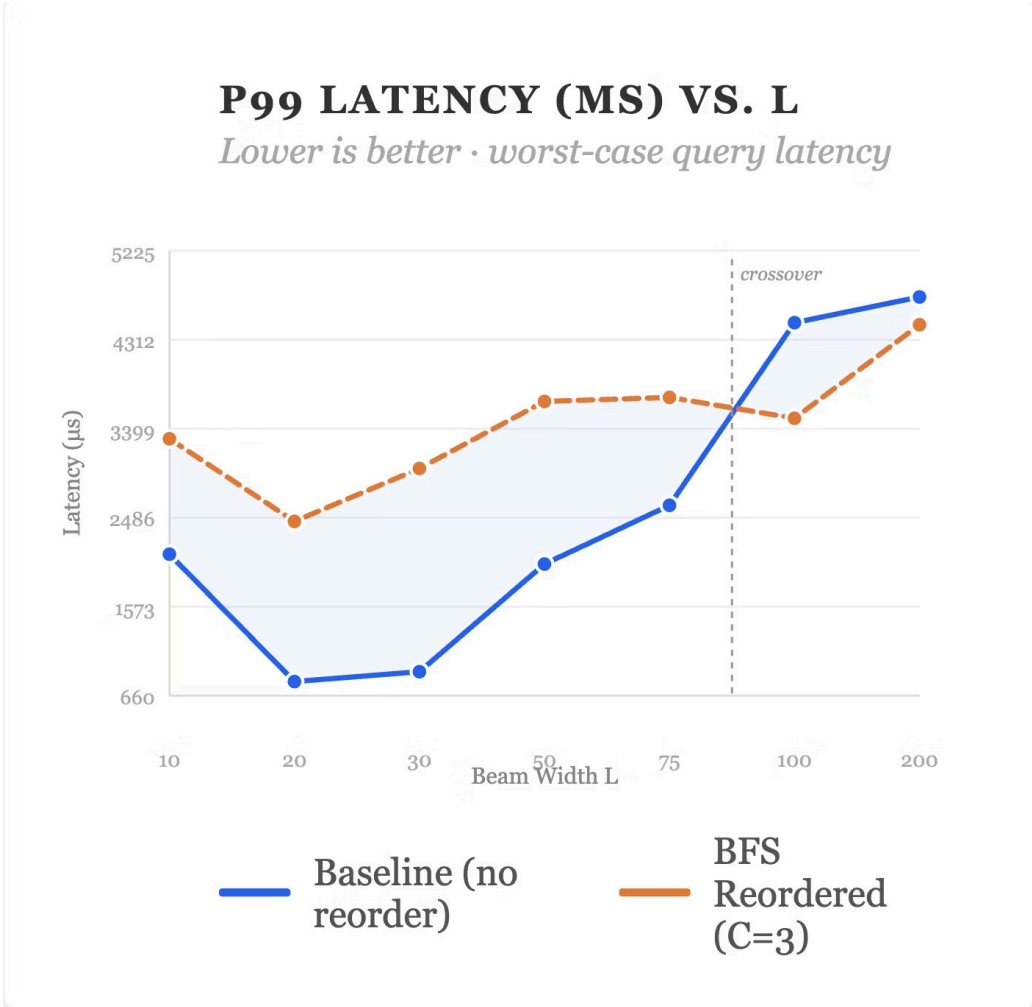
Result: Lowered P99 Latency at L = 10, 20, 30. Converged for larger search sizes.

For larger L values, since graph has good connectivity, destination is found in almost the same time regardless of starting point.

Start everywhere VS Start from the right place!

BFS-Based Node Reordering (*2024 by Coleman et al.*)



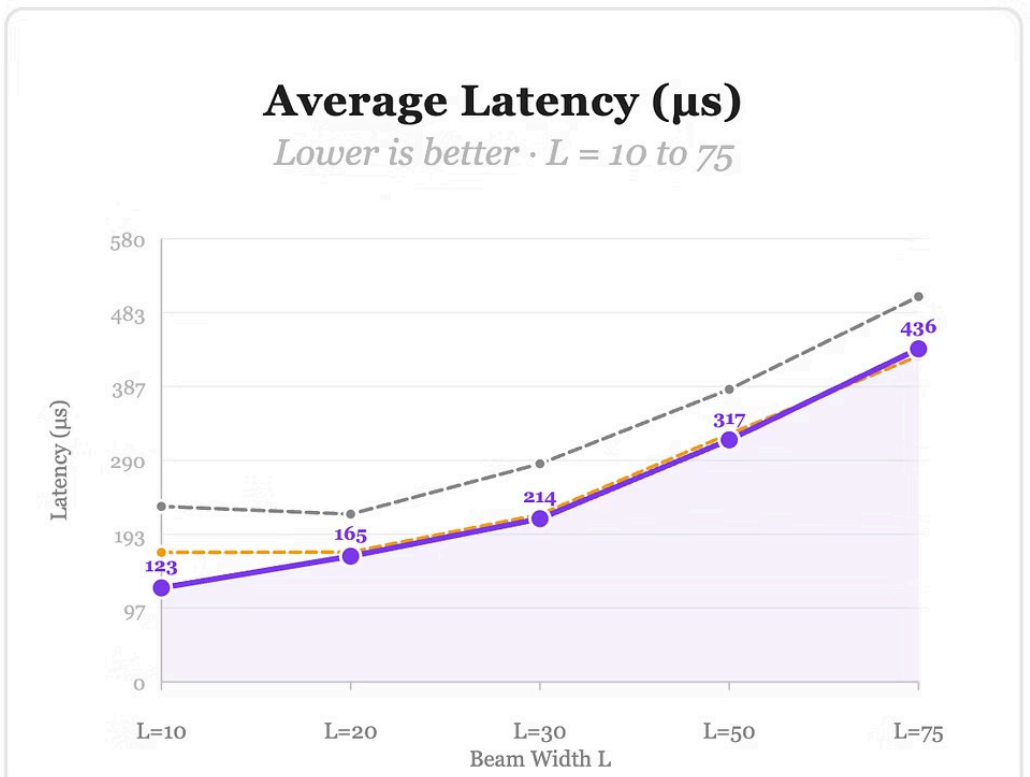


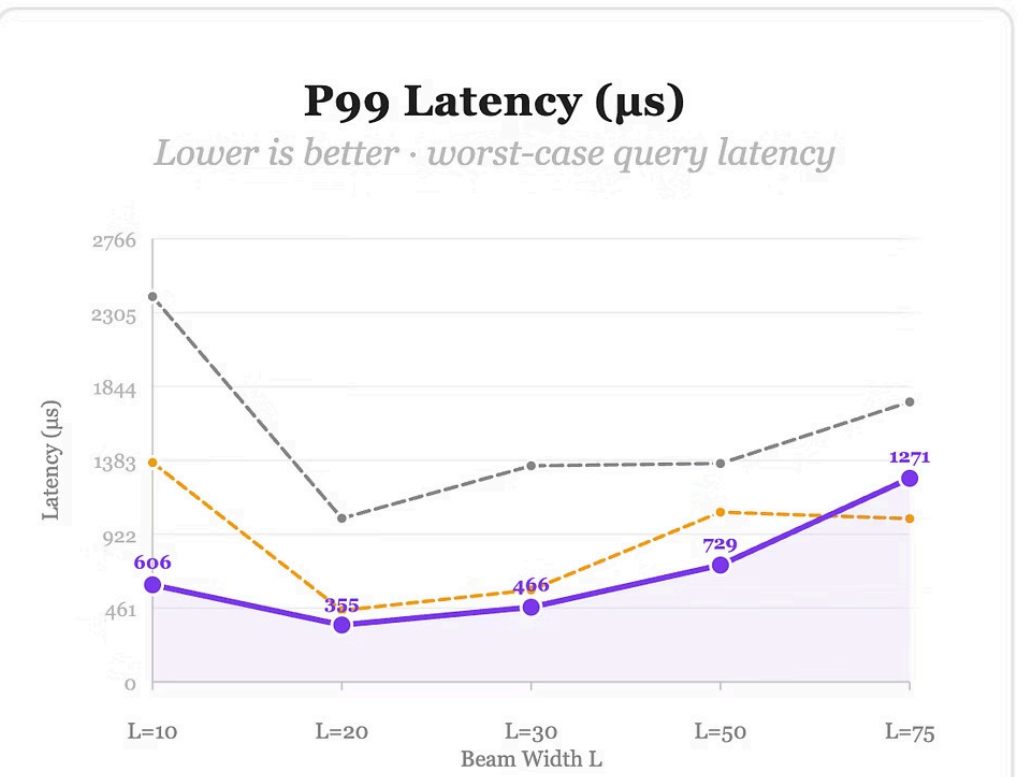
VSAG: Optimized Search Framework for ANNs (*Alipay 2025*)

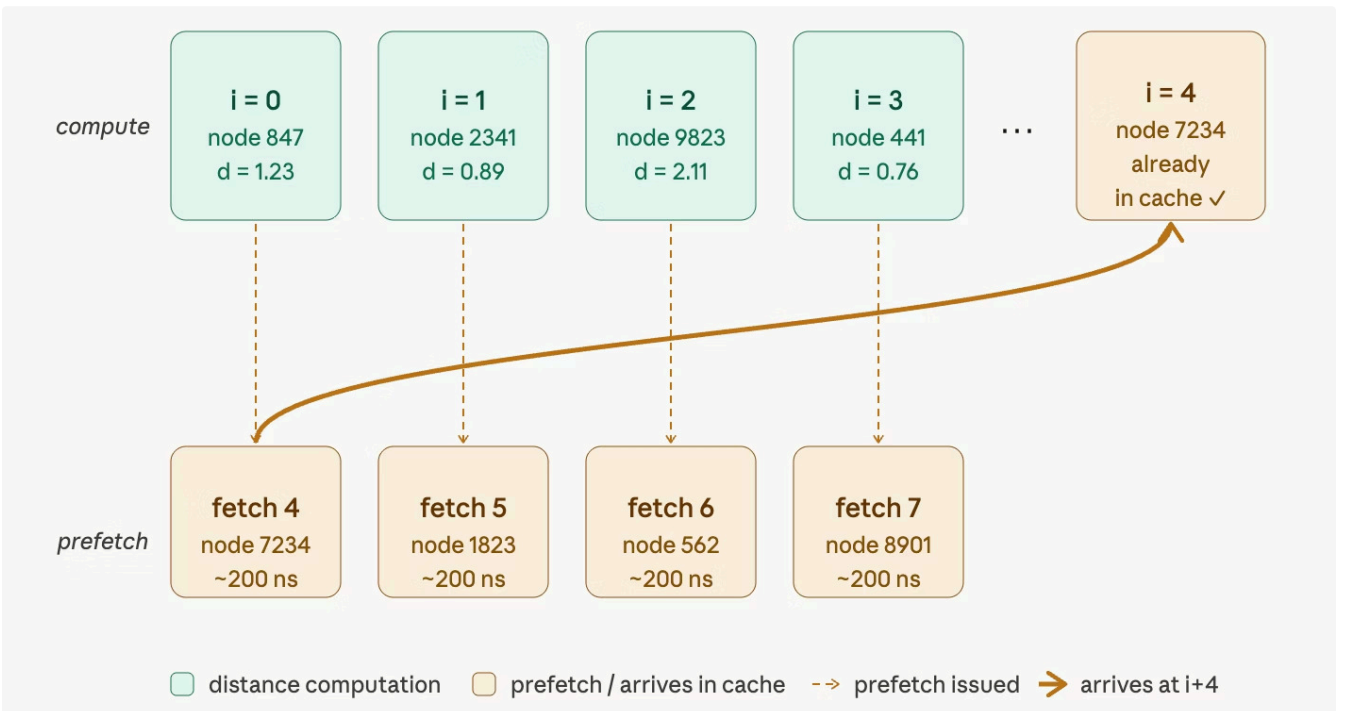
Original Baseline (before A–D)

Proposals A–D Quantized

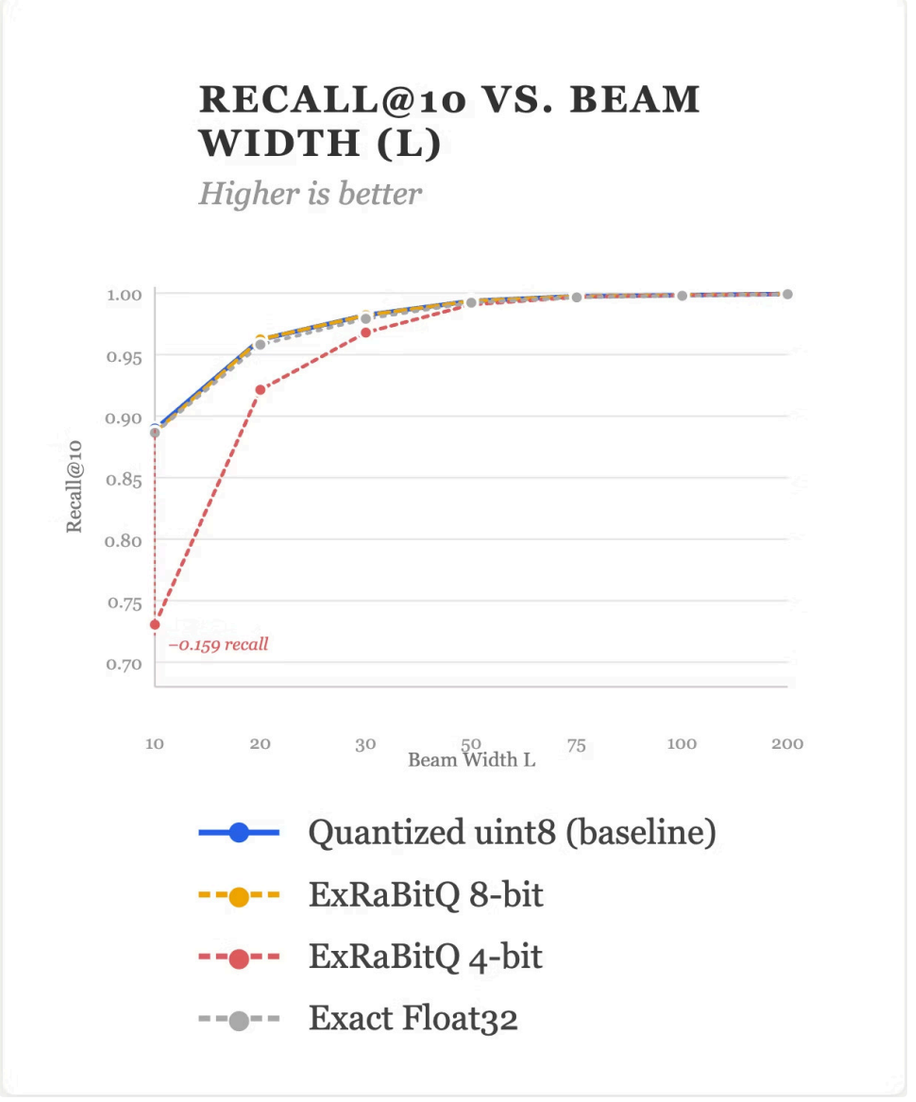
Quantized + Stride Prefetch (new)

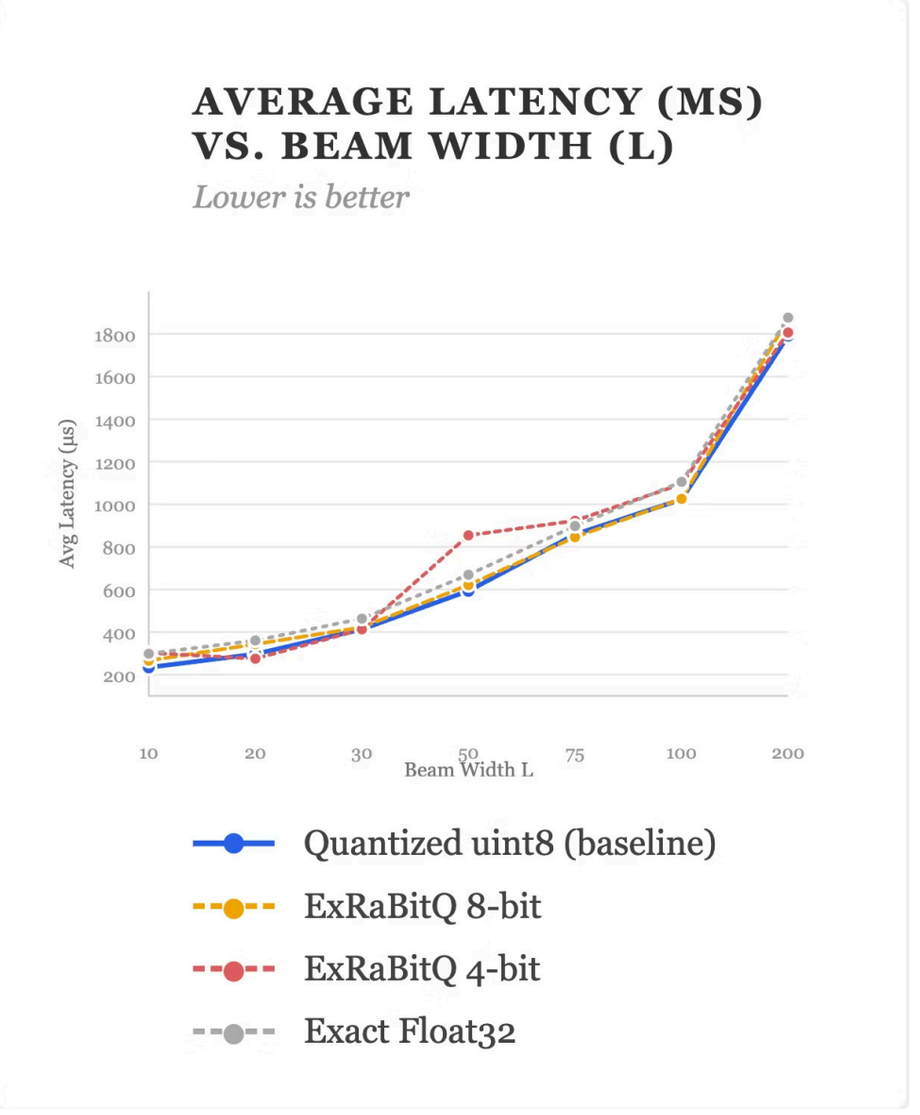






Extended RaBitQ (*2024 by Gao et al.*)





HNSW Analysis on SIFT1M

Hierarchical Navigable Small World vs DiskANN Implementation

HNSW Implementation Details

- Multi-layer Graph Structure with logarithmic search complexity
- Probabilistic Layer Assignment using geometric distribution
- Greedy Multi-layer Descent from global entry point
- Beam Search with dynamic candidate frontier
- Heuristic Neighbor Selection for edge diversity
- Bidirectional Edge Maintenance with degree-constrained pruning

HNSW Summary (Best Relevant Configurations)

Configuration	Recall@10	Avg Latency (µs)	Avg Dist Cmps
M=8, ef=100	0.9491	622.7	248.0
M=16, ef=200	0.9789	1129.6	448.0

HNSW vs Vamana: Head-to-Head

HNSW

- Recall@10: 0.9909 (ef=400)
- Avg Latency: 1129.6 µs
- Distance Computations: 448
- Strength: Logarithmic hierarchy

Vamana Best

- Recall@10: 0.9936 (R=64, L=50)
- Avg Latency: 521.7 µs
- Distance Computations: ~180
- Strength: Flat navigable graph

Key Findings

- HNSW M=16 is 7x more efficient in distance computations (265 vs 1905) to reach 97-98% recall.
- Vamana R=32 maintains lower latency at high recall due to cache optimization.
- Vamana R=64 with ADC achieves highest recall (99.9%+) for high-accuracy requirements.
- Vamana is 2.2x faster at equivalent recall.
- Trade-off: HNSW excels in computation efficiency; Vamana excels in latency and recall ceiling.

AI Tools: Effective vs. Failed

Where Claude/Gemini Helped

- Boilerplate: CLI parsing, I/O, CMakeLists
- Algorithm explanation: α-RNG pruning, beam search
- Debugging, locking and memory issues in parallel build, and optimizing for C++ on a hardware level
- Early-abandonment proposition: using parallelization with ADX
- Literary review and search for implementations
- Merging repositories across team members effectively
- Handling cross-efficiency with Windows and Mac
- Caveman optimization (short responses for token optimization) and system prompts help
- AI cross-validation

Where AI Failed

- RaBitQ was mistakenly theorized to reduce distance computations by decorrelating dimensions, but we realized the results were sub-par
- Easily swayed by prompt suggestions (prompts have an imbalanced effect on final ideas, and commitment to them); too unwilling to disagree
- Forgot to remap vectors to indices on disk after mapping on memory (BFS-Based Node Reordering)
- Hallucinates results with fake benchmark numbers

⚠ **Key lesson:** AI cannot predict performance effectively, and tends to be too ambitious. Ideas usually require human input to refine, and AI tools to implement

Conclusion & Key Takeaways

1 Build quality dominates search-time optimization

Proposals A–D cut latency ~30%. Build-time fixes (medoid, R=64, full visited set) achieved **46% lower avg latency and 78% lower P99** at equivalent recall.

2 Mean vs. P99 latency measure different things

Medoid initialization specifically eliminates hard queries, improving P99 by up to 78%, far exceeding mean improvements.

3 Negative results are scientifically valuable

FINGER, PCA traversal, multi-entry-point search all failed, revealing SIFT1M's high intrinsic dimensionality and lack of cluster structure.

View Repository on GitHub